Brainrot Breakout: A Browser-Based Serious Game for Digital Well-Being Using Hand-Made Data Structures

Samuel Navarro, José Daza, Manuel Hormechea, Iván Rivas
Último Scroll Studios, Universidad del Norte
Barranquilla, Colombia

November 12, 2026

Abstract

Excessive social media engagement and doomscrolling have raised concerns about digital well-being, while many interventions rely on reminders or explicit instruction. This work presents *Brainrot Breakout*, a browser-based two-dimensional serious game that represents the trade-off between short-term relief and compulsive phone use through gameplay. The objective was to develop an accessible game that promotes reflection on healthier digital habits while applying data structures and abstract data types implemented from scratch. The project was developed in Godot 4.7 using GDScript and exported for web deployment. Its mechanics combine randomly positioned enemies, skill-based combat, six zones with boss encounters, accessibility settings, persistent records, and a leaderboard. Four hand-made abstract data types were implemented: `Lista`, `Cola`, `Leaderboard`, and `Registro`; the implementation uses linked-list, queue, and ordered-ranking structures. The software and its core mechanics were checked through code inspection. No empirical usability or performance outcome is claimed. The project demonstrates how data structures can support an interactive digital-well-being metaphor.

Keywords— serious games, digital well-being, doomscrolling, data structures, abstract data types, accessibility, Godot

1 Introduction

Modern digital ecosystems are designed to sustain continuous engagement. Public health authorities have highlighted potential risks associated with social media exposure and youth mental health [1]. Platform features such as infinite feeds and algorithmic content delivery may reinforce habitual use and problematic engagement [2]. Doomscrolling, understood here as compulsive consumption of distressing digital content on mobile devices, is one manifestation of this pattern.

The problem addressed in this work is the limited availability of serious games that approach excessive social media use through implicit, mechanics-driven awareness rather than lectures or direct questions. Although usage limits and behavioral prompts can reduce time spent in selected applications, their effects on broader problematic-use measures may be limited [3]. Serious games provide an alternative setting in which players can experience behavioral trade-offs through interaction [4]. Accordingly, this project presents *Brainrot Breakout*, a two-dimensional action game developed by Último Scroll Studios for the Data Structures I course at Universidad del Norte. Its central mechanic makes the virtual phone a source of short-term health recovery while increasing a dopamine meter and reducing visibility. The objective is to develop an accessible browser-based game that encourages reflection on healthy digital habits while integrating custom data structures and abstract data types into its architecture.

The remainder of this article is organized as follows. Section 2 presents the contributions, Section 3 reviews related work, and Section 4 describes the methodology, including the system design, the data structures, the technological environment, and the use of AI tools.

2 Contributions

The contributions of this work are:

- **A playable metaphor for digital compulsion:** a risk–reward loop in which opening the phone restores health while increasing dopamine and blurring the screen.
- **A six-zone action game:** six distinct boss encounters represent platform-related dynamics and require observation, timing, and tactical movement.
- **Hand-made data abstractions:** four abstract data types (`Lista`, `Cola`, `Leaderboard`, and `Registro`) support wave progression, attack rotation, rotation of the pause-menu hints, ranking, and session persistence.
- **Inclusive interaction options:** color-vision filters, independent audio buses, video blocking, two

control schemes, and automatically activated touch controls.

- **A browser-deployable implementation:** the game was exported from Godot for web execution, with local persistence for settings, records, and leaderboard entries.
- **A documented requirements framework:** functional and non-functional requirements organized with FURPS+ guide the implementation and review of the game.

3 Related Work

Digital interventions targeting excessive social media consumption and problematic digital habits have gained significant scientific attention. The United States Surgeon General issued an advisory highlighting how unchecked social media exposure poses substantial risks to youth mental health, associating heavy daily usage with higher rates of depressive symptoms and anxiety [1]. To synthesize existing empirical findings, Sharpe et al. [2] conducted a systematic scoping review on doomscrolling, demonstrating consistent cross-sectional associations between compulsive negative media consumption and elevated distress, while emphasizing the critical scarcity of evaluated preventive interventions.

In response to these risks, several researchers have designed software-based interventions. Mertens et al. [3] evaluated *Wellspent*, an iPhone application implementing behavioral nudges and self-set usage limits. In a three-week randomized controlled trial with 70 participants, the authors observed a statistically significant reduction of 29 minutes per day in target application usage; however, the intervention did not produce significant reductions in broader self-reported problematic usage scale scores. Building upon this trial dataset, Brockmeier et al. [5] performed a secondary analysis examining psychological mechanisms, finding that stress reduction occurred primarily among users exhibiting higher baseline levels of problematic engagement. Taking an educational prevention approach in vocational schools, Pietsch et al. [6] conducted a cluster-randomized controlled trial evaluating the mobile application *Meine Zeit ohne*. Their study demonstrated that voluntary two-week digital media abstinence commitments fostered measurable improvements in student health awareness across large cohort groups.

Alternative approaches leverage interactive narratives and serious games to promote digital well-being. Andrew et al. [4] conducted a systematic scoping review mapping serious health promotion games for adolescents, confirming their efficacy in transforming health attitudes while identifying a lack of gamified interventions addressing digital addiction. Specifically targeting depressive symptoms, Sriwatanathamma et al. [7] developed *Blue-Line*, a visual novel integrating Cognitive Behavioral Therapy (CBT) through the ABCDE model. The game constrained excessive virtual social interactions to sessions under one minute to simulate healthy boundaries, demonstrating how interactive narratives can deliver implicit therapeutic lessons.

Among the works reviewed here, a clear research gap remains. Existing interventions rely primarily on external restrictive prompts, retrospective self-reporting, or linear narrative choices. None of the reviewed systems integrates a real-time 2D action game loop where smartphone interaction functions as a risk-reward mechanic, coupled with implicit digital health awareness, inclusive visual design for colorblindness, and direct accessibility through standard web browsers.

4 Methodology

4.1 Solution Alternatives

Three alternatives were considered for the implementation:

- *Alternative A: Python, Pygame, and pygbag.* This option offered a familiar language and game-library ecosystem, but required additional work for interface components, shaders, and touch input.
- *Alternative B: Java and libGDX.* This option offered a structured programming environment and game-development facilities, but web deployment and custom interface work introduced additional implementation effort.
- *Alternative C: Godot 4 and GDScript.* This option provided a game engine with web export, shader support, audio buses, and an integrated interface system [8].

4.2 Evaluation and Selection

The alternatives were compared using six criteria and weights. Table 1 reports the scores recorded by the project team; the weighted totals are reproduced from that evaluation and have not been independently benchmarked. Alternative C was selected because it received the highest recorded weighted score and provided the web, shader, audio, and interface facilities required by the project.

4.3 Functional Requirements

Table 2 presents the functional requirements used to guide implementation.

Table 1: Weighted evaluation of solution alternatives (scale 1–5).

Criterion	Weight	A	B	C
Web and touch support	0.25	3	2	5
Ease of implementation	0.20	4	3	4
Performance	0.15	3	4	4
Maintainability	0.15	3	4	4
Development time	0.15	4	2	4
Built-in UI, audio, and shaders	0.10	2	2	5
Weighted total	1.00	2.85	2.80	**4.35**

4.4 Non-Functional Requirements

Non-functional requirements were organized using FURPS+ [9]. Table 3 summarizes the categories and constraints.

4.5 Requirements Evaluation and Method

The team used the functional and non-functional requirements as implementation and review criteria. The development process used GitHub Flow, pull requests, and a Trello task board; details are provided in Section 4.7. The game was implemented in Godot 4.7 using GDScript. The requirements were checked through code inspection and gameplay evidence, without empirical usability or performance measurements. The FURPS+ categories cover functionality, usability, reliability, performance, supportability, and additional constraints. Nielsen's usability heuristics provide a complementary basis for interface inspection [10].

A serious-game approach is consistent with research on game-based health promotion [4]. The project did not claim a measured health outcome; its intended contribution was an interactive representation of the trade-off associated with repeated phone use.

4.6 System Design and Data Structures

The use cases of the system are modeled in Fig. 1. The only actor is the player, who interacts with the main menu to start a new game (Zones 1 to 6 in sequence), to play a single zone in the arcade mode, to change the settings (volume, color-blindness filter, phone videos, and controls), to read the help, to view the leaderboard, and to exit the game (this option is hidden in the web build). Starting a new game and choosing an arcade zone both include *Play zone*, which in turn includes *Slap zombies*, the basic attack of the game. Four use cases extend *Play zone*: opening the phone, pausing the game, retrying the zone after losing (the death screen), and fighting the boss, which appears every five waves. Saving a score extends the boss fight, because the nickname for the leaderboard is requested only after the final boss is defeated.

The architecture is organized in components, as Fig. 2 shows. The main menu loads a zone scene, and each of the six zones (Table 4) is composed of a map, the player, a heads-up display (HUD), and a wave generator; the root script of the zone launches the waves and summons the boss. Five global singletons (autoloads) are available to every scene: `Ajustes` holds the volume buses (master, music, and effects), the color-blindness filter, the blocking of the phone videos, and the control scheme (arrows plus X, or WASD plus right click), and stores them in `ajustes.cfg`; `Pausa` manages the pause menu and its rotating hints; `Transicion` manages the fade transitions between zones; `MusicaGlobal` manages the music; and `Muerte` shows the death screen, which freezes the game and lets the player retry the zone or return to the menu. The player reads and writes the personal record and the elapsed time through `Registro`, and the menu displays the `Leaderboard`.

The bosses are the link between the zones and the data structures, and Fig. 3 shows their classes together with those of the data structures. A boss receives damage only while it is stunned and enters a second phase when its health drops below a threshold (Table 4). The bosses of Zones 3 to 6 inherit from the base class `JefeBase`, which implements three states (intro, attacking, and stunned) and an attack queue with cooldowns. The bosses of Zones 1 and 2 have their own logic and follow the same contract with the rest of the game (the signals `vida_cambiada` and `murio` and the method `recibir_golpe`); for this reason the figure links them to `JefeBase` with a dashed arrow instead of an inheritance arrow. The final boss displays the victory screen and requests the name for the leaderboard. All graphics were drawn by code as placeholders that can be replaced by sprites.

The data structures were implemented by hand, without using the `Array` type of GDScript to store their elements; the evidence is in the source files `lista.gd`, `cola.gd`, `leaderboard.gd`, and `registro.gd` (`https://github.com/Moon-collab1/Brainrot-Breakout`). Table 5 summarizes the structures, their use in the game, and the cost of their main operations. Arrays and dictionaries still appear in the game only as configuration data, for example the initial wave sizes, which are loaded into the wave queue, and the cooldown tables of the bosses.

Singly linked list. `Lista` is a singly linked list ADT (`lista.gd`). Each element lives in a node (inner class `Nodo`, with the fields `dato` and `sig`), and the list keeps a pointer to the first node and a size counter. Its operations are append at the end (`agregar_final`), remove the first element (`quitar_primero`), get by position without removing (`obtener`), and empty check (`esta_vacia`). Since the list has no pointer to the last

Table 2: Functional requirements of Brainrot Breakout.

ID	Requirement description
FR-01	The system shall provide menu access to the game, arcade mode, settings, leaderboard, and help.
FR-02	The player shall move and execute a slap attack (X key or right-click) that frees zombies within a frontal circular area; the freed count determines the score.
FR-03	The player shall toggle a virtual phone: while open, health shall increase by 5 HP/s and dopamine by 8/s, movement shall slow, and the screen shall blur; while closed, dopamine shall decrease by 2/s.
FR-04	The player shall lose and restart the current zone when health reaches zero or dopamine reaches 100.
FR-05	The game shall randomize zombie spawn positions along the left, right, and bottom borders and the phone video content.
FR-06	Gameplay shall require slap aiming, attack dodging, and decisions about when to open the phone.
FR-07	The game shall provide six zones, progressive waves, and a boss every five waves, with stun vulnerability and phase transitions.
FR-08	The system shall maintain a persistent leaderboard sorted by freed zombies and then by shorter completion time.
FR-09	The system shall persist settings, the personal record, and leaderboard entries in local files.
FR-10	The system shall provide color-vision filters, independent volume buses, video blocking, and two control schemes.
FR-11	The system shall provide help screens, boss attack banners, and pause menus.
FR-12	The game shall activate on-screen touch controls on touch-enabled devices.
FR-13	The game shall provide audio feedback for gameplay, transitions, and victory.
FR-14	Gameplay mechanics and states shall be represented graphically.
FR-15	The core logic shall use at least three data-structure types and two hand-made abstract data types.

Table 3: Non-functional requirements structured using FURPS+.

Category	ID	Description
Functionality	NFR-F1	The game shall run locally without an external application server and shall store player nicknames locally.
Usability	NFR-U1	Instructions and contextual guidance shall be accessible through the interface.
Usability	NFR-U2	Audio-visual feedback shall communicate gameplay states.
Reliability	NFR-R1	The game shall handle missing or malformed persistent-file entries without terminating the session.
Reliability	NFR-R2	Nicknames shall be sanitized before persistence.
Performance	NFR-P1	Browser frame rate shall be measured on desktop and mobile devices.
Supportability	NFR-S1	Source files shall use modular organization, Spanish identifiers/comments, and a header comment.
Supportability	NFR-S2	Version control shall follow GitHub Flow with pull-request integration.
Constraints (+)	NFR-C1	The game shall be 2D, developed in Godot 4.7, exported for the web, and avoid explicit instructional questions.

Table 4: Zones and bosses of the game. Wave: wave at which the boss appears. HP/P2: boss health and health threshold of its second phase.

Zone	Setting	Boss	Wave	HP/P2
1	Street	El Influencer Viral	5	56/28
2	Social building	El Algoritmo	10	60/30
3	Loading area	Capataz	15	50/25
4	Elevator	Gemelo	20	40/20
5	Laboratory	Dr. Dopamina	25	24/12
6	Supercomputers	Núcleo-∞	30	25/10

node, appending traverses the whole list and costs $O(n)$; removing the first element costs $O(1)$; getting the element in position i costs $O(i)$; and the empty check costs $O(1)$.

Queue. `Cola` is a queue ADT (`cola.gd`) built also with linked nodes, but it keeps pointers to the first node (`frente`) and to the last node (`fin`). For this reason `encolar`, `desencolar`, and `ver_frente` cost $O(1)$, and `fin` is reset when the last element leaves. In the game, `Pausa` uses `Cola` to rotate the hints shown in the pause menu: the hints are enqueued in random order, one leaves the queue each time the game is paused, and the queue is refilled in a new random order when it becomes empty.

Lista used as queue and as plain list. In the rest of the game, the queues are built over `Lista`, appending at the end and removing from the front, as the header of `lista.gd` describes. The wave generator keeps the sizes of the next waves: the queue starts with three sizes (5, 8, and 12 in Zone 1, with a step of 4; from 14, 16, and 18 up to 30, 32, and 34 in Zones 2 to 6, with a step of 2), each wave removes the first size and appends the last size plus the step, and the wave counter continues

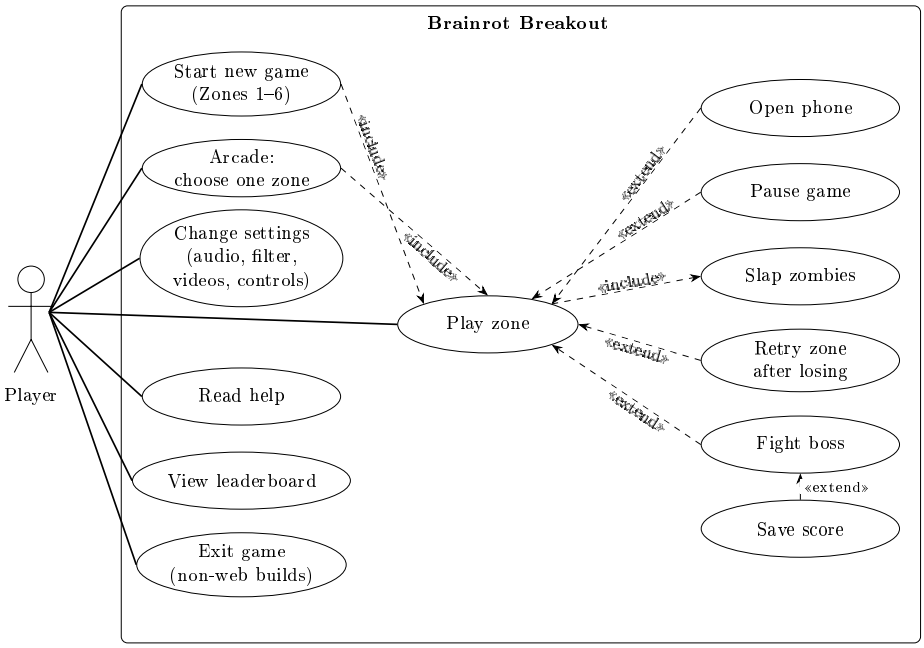


Figure 1: Use case diagram of Brainrot Breakout. Dashed arrows point from the including or extending use case as UML prescribes: «include» from the base to the included case, «extend» from the extension to the base.

between zones, which places a boss at waves 5, 10, ..., 30. The same structure rotates the attacks of the six bosses, each attack having a cooldown (a front attack that is still cooling down, or whose firing condition is not met, goes back to the end of the queue and the next one is tried), keeps the boxes of Zone 3 (the oldest one leaves first), and keeps the position history used by a delayed attack of Zone 4. As a plain list, `Lista` stores the antennas of Zone 1 and the badge batches of Zone 2.

Using `Lista` as a queue when `Cola` offers an $O(1)$ enqueue is a design trade-off. Asymptotically, `Cola` is the better choice for a pure queue. In the source code, however, the header of `lista.gd` already documents `Lista` as the queue of the waves and of the boss attacks, while `Cola` is used only by `Pausa`; keeping a single structure for the gameplay queues kept that code uniform. The cost is small in this game: the wave queue always holds three sizes, the attack queues hold a handful of attacks (at most about seven), the boxes of Zone 3 at most ten, and the position history of Zone 4 about 21 positions (one sample every 0.05 s with a delay of 1 s), so one append traverses at most about 21 nodes. This cost was derived from the code and not measured. Replacing `Lista` with `Cola` in the pure queues will be evaluated as future work.

Leaderboard. `Leaderboard` (`leaderboard.gd`) keeps a `Lista` of at most ten entries (nickname, freed zombies, and time), always ordered from best to worst: more freed zombies first and, in case of a tie, the shorter time. Because `Lista` has neither an iterator nor an insertion at a position, each insertion builds a new list: it traverses the current one with `obtener(i)`, places the new entry before the first entry that it beats, and copies the first ten entries into a second list. Each `obtener(i)` costs $O(i)$ and each append costs $O(n)$, so one insertion costs $O(n^2)$ in the number of entries; since $n \leq 10$ (eleven before the cut), this cost is bounded by a constant. Saving also calls `obtener(i)` in a loop, and loading appends one line at a time, so both cost $O(n^2)$ as well. The list is written to `leaderboard.txt`, one line per entry with the format `name;freed;time`. Nicknames are sanitized before the insertion: they are stripped, the character `;` and line breaks are removed, they are cut to 12 characters, and `ANONIMO` replaces an empty name. When the file is loaded, lines with a wrong number of fields or non-numeric values are skipped, so a damaged line does not stop the game. The loader, however, trusts the order and the size stored in the file, so a file edited by hand could break the sorted order; this is a limitation of the current implementation.

Session state and record. `Registro` (`registro.gd`) holds the state that survives scene changes in static variables, which any script reads without creating an instance: the freed count with which the player enters the current zone, the arcade-mode flag, and the elapsed time of the game. The player adds the elapsed time at every physics frame, and the leaderboard uses it as the tie-breaker. `Registro` also manages the personal record: `leer` returns the record stored in `record.txt` (zero if the file does not exist or cannot be opened), and `guardar` writes a new value only if it exceeds the stored one.

Together, these structures meet requirement FR-15: a singly linked list, a queue, and an ordered list (the ranking) as data structure types, four ADTs (`Lista`, `Cola`, `Leaderboard`, and `Registro`), and three text files that

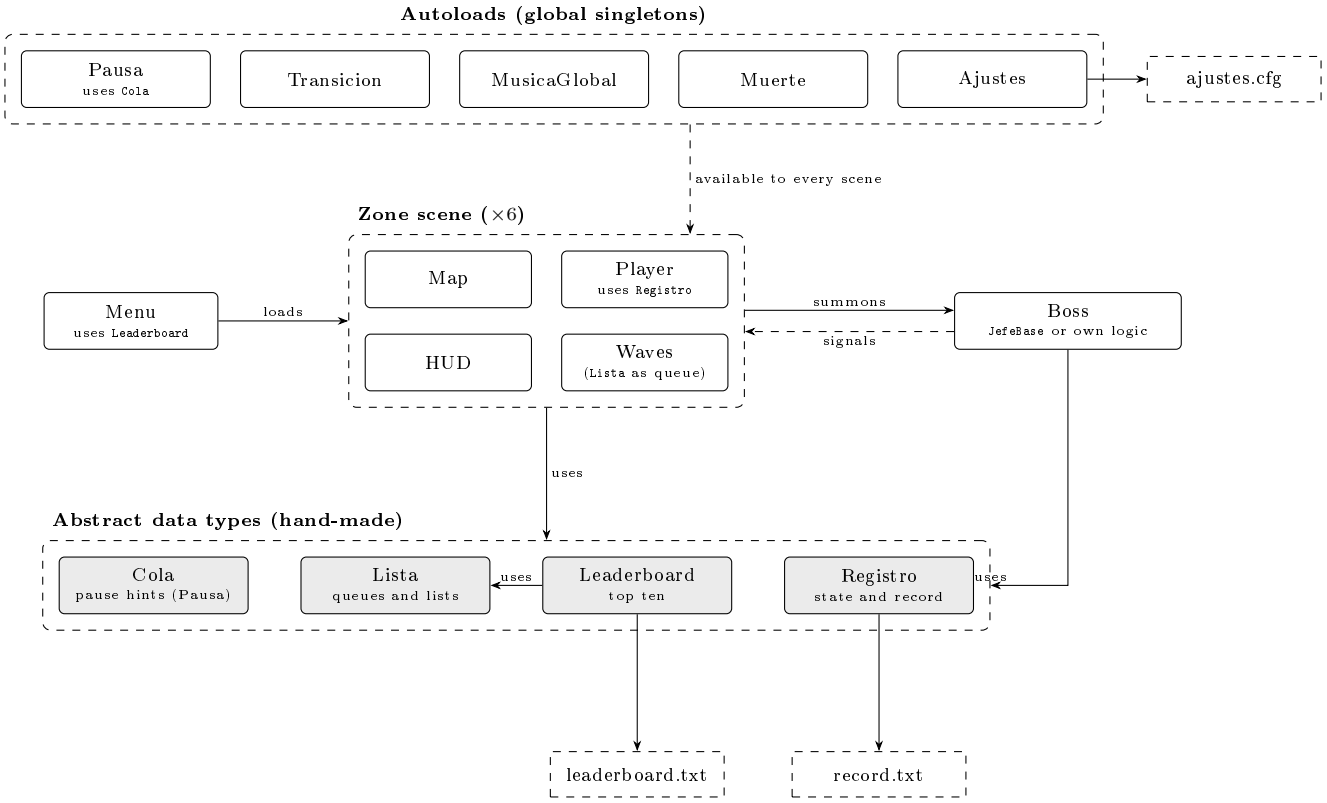


Figure 2: Component architecture of Brainrot Breakout. Solid arrows indicate use or control flow, dashed arrows indicate global availability or signals, shaded boxes are the hand-made abstract data types, and dashed rectangles are files.

persist the data.

4.7 Technological Environment and Development Process

Table 6 lists the technological choices of the project, each with its justification. The game was implemented in Godot 4.7 with the GDScript language and the GL Compatibility renderer, with a base resolution of 1152×648 pixels. Two shaders were written: the color-blindness filter (protanopia, deuteranopia, and tritanopia) and the vignette shader used with the phone, which also produces the blur of the screen while the phone is open. The game was exported to the web (HTML5), with touch controls that activate automatically on touch screens, and it was published on itch.io (`https://lookthereis.itch.io/brainrot-breakout`).

The source code is hosted in a public GitHub repository (`https://github.com/Moon-collab1/Brainrot-Breakout`) and was managed following GitHub Flow [11]: each change was developed in a branch and integrated through a pull request. The tasks were managed in a Trello board, with tasks assigned by role (Table 7). The code was written with names and comments in Spanish and a header comment in every file.

4.8 Use of Artificial Intelligence Tools

The tool used was Claude, an AI assistant developed by Anthropic. It was used for the following purposes: to propose ideas for the bosses of the game, to draft and revise this article, to produce the use case diagram, to resolve problems that appeared when branches were merged in pull requests, and to generate some parts of the game code. The team reviewed the proposals and tested the code in the game before integrating it, and the authors are responsible for the final content of the game and of this article.

References

[1] Office of the Surgeon General, "Social media and youth mental health: The U.S. Surgeon General's advisory," 2023.

[2] B. T. Sharpe, I. Tyndall, D. R. Poulus, and E. A. Obine, "The influence of doomscrolling on mental health: a scoping review," *Mental Health and Digital Technologies*, vol. 3, no. 3, pp. 290–342, 2026.

[3] L. Mertens, L. C. Brockmeier, C. Roitzheim, T. Radtke, T. Dingler, and J. Keller, "Promoting self-regulated social media use on smartphones with

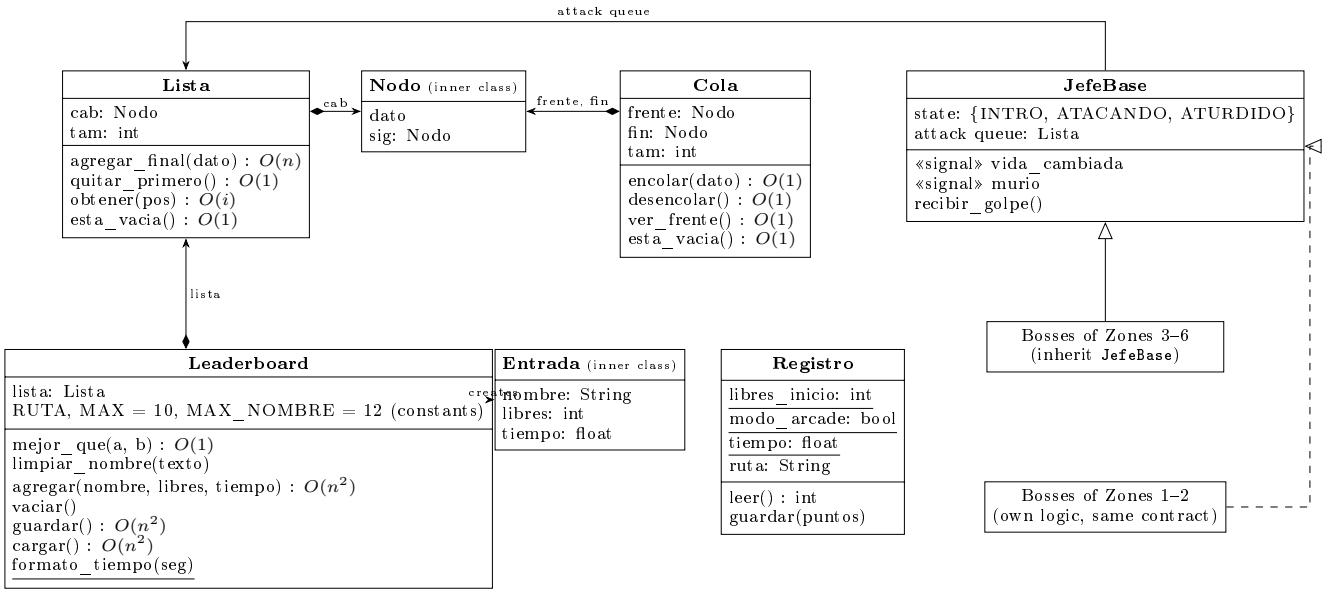


Figure 3: Class diagram of the hand-made data structures and of the boss base class. Diamonds mark composition, the hollow triangle marks inheritance, and the dashed arrow marks a shared contract without inheritance. Underlined members are static, and the costs correspond to the implementation in the source files.

a mobile intervention app (Wellspent): Randomized controlled trial," *JMIR mHealth and uHealth*, vol. 14, p. e56824, 2026.

[4] N. Andrew *et al.*, "Serious games for health promotion in adolescents – a systematic scoping review," *Education and Information Technologies*, vol. 28, no. 5, pp. 5519–5550, 2023.

[5] L. C. Brockmeier, L. Mertens, C. Roitzheim, and J. Keller, "Effects of an intervention targeting social media app use on well-being outcomes: A randomized controlled trial," *Applied Psychology: Health and Well-Being*, vol. 17, no. 1, p. e12646, 2025.

[6] B. Pietsch *et al.*, "Effects of an app-based intervention program to reduce substance use, gambling, and digital media use in adolescents and young adults: A multicenter, cluster-randomized controlled trial in vocational schools in germany," *International Journal of Environmental Research and Public Health*, vol. 20, no. 3, p. 1970, 2023.

[7] P. Sriwatanathamma, V. Sirivesmas, S. Simatrang, and N. H. Bhowmik, "Gamifying cognitive behavioral therapy techniques on a mobile visual narrative serious game (BlueLine) to reduce depressive symptoms in young adults: Design and usability study," *JMIR Serious Games*, vol. 11, p. e41638, 2023.

[8] Godot Engine Contributors, *Godot Engine Documentation*, 2024.

[9] R. B. Grady and D. L. Caswell, *Software Metrics: Establishing a Company-Wide Program*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1987.

[10] J. Nielsen, "10 usability heuristics for user interface design," 1994.

[11] GitHub Guides, "Understanding the GitHub flow," 2022.

Table 5: Data structures and abstract data types used in the game.

Structure / ADT	**Use in the game**	**Main operations and cost**
Singly linked list `Lista` (`lista.gd`)	Base structure of the queues and lists below	Append at end $O(n)$; remove first $O(1)$; get by position $O(i)$; empty check $O(1)$
`Lista` as queue	Sizes of the next waves; attack rotation of the six bosses; boxes of Zone 3; position history of Zone 4	Enqueue (append at end) $O(n)$; dequeue (remove first) $O(1)$; at most about 21 elements
`Lista` as plain list	Antennas of Zone 1; badge batches of Zone 2	Same operations and costs as `Lista`
Queue `Cola` (`cola.gd`)	Rotation of the pause-menu hints in `Pausa`	Enqueue, dequeue, peek, and empty check $O(1)$
ADT `Leaderboard` (`leaderboard.gd`)	Top-ten ranking over an ordered `Lista`	Ordered insertion $O(n^2)$ with $n \leq 10$; save and load $O(n^2)$; nickname sanitizing; damaged lines skipped
ADT `Registro` (`registro.gd`)	Session state and personal record	Read the record; save it only if it is exceeded (one number per file)
Text files	`ajustes.cfg`, `record.txt`, and `leaderboard.txt`	Persist settings, record, and leaderboard (lines `name;freed;time`)

Table 6: Technological choices and their justification.

Aspect	**Choice**	**Justification**
Engine and language	Godot 4.7 with GDScript	The engine provides native web export, touch input, shaders, audio buses, and a user interface toolkit [8], and GDScript has a syntax close to Python, a language familiar to the team.
Renderer	GL Compatibility	It is the renderer that Godot 4 supports for web export [8].
Resolution	1152×648 pixels	A 16:9 base size, the default of Godot, gives one fixed reference for the layout of all scenes and of the HUD.
Visual effects	Shaders (color-blindness filter, vignette, and blur)	Shaders apply effects to the whole screen on the GPU without changing the game objects or requiring extra art.
Deployment	Web export (HTML5) and itch.io	The course requires that the game runs in a browser, and itch.io hosts HTML5 games under a public link.
Version control	GitHub with GitHub Flow and pull requests [11]	Short-lived branches let four members work in parallel, and each pull request makes a change visible to the team before it is merged.
Task management	Trello board with tasks by role	A shared board shows every task and its owner, and assigning tasks by role (Table 7) avoids overlapping work.
Code conventions	Names and comments in Spanish; header comment in every file	A common convention keeps the code written by four authors consistent and easy to read (NFR-S1).

Table 7: Team roles of Último Scroll Studios.

Member	**Role**
Samuel Navarro	Project manager and testing director
José Daza	Design director
Iván Rivas	UI/UX director
Manuel Hormechea	Documentation director